Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

«Запросы на выборку и модификацию данных. Представления. Работа с индексами» по дисциплине «Проектирование и реализация баз данных»

Вариант 2. БД «Сессия»

Обучающийся Левчук Софья Александровна
Факультет прикладной информатики
Группа K3241
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии
Преподаватель Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург
2025/2026

Цель работы: овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Оборудование: компьютерный класс.

Программное обеспечение: СУБД PostgreSQL, pgadmin 4.

Практическое задание:

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) **с использованием подзапросов**.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

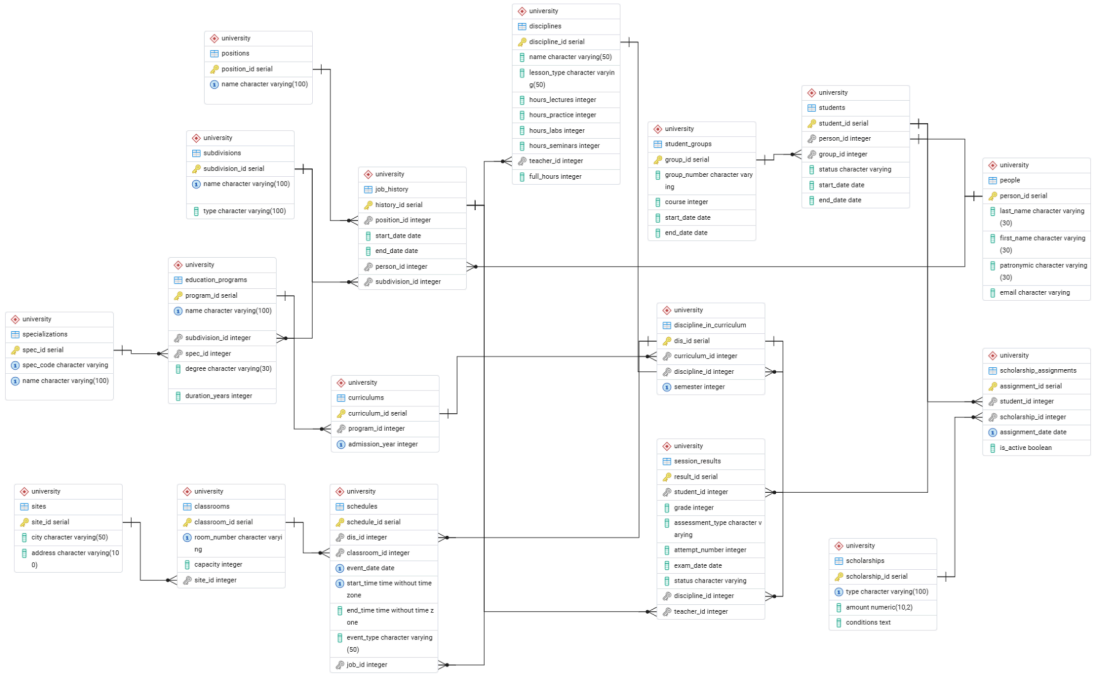

Рисунок 1 – Схема логической модели базы данных, сгенерированная в Generate ERD

Создание запросов

1. Составить список дисциплин, которые должны быть сданы заданной группой с указанием дат сдачи и фамилий преподавателей.

```
SELECT
   d.name AS discipline_name,
   sch.event_date AS exam_date,
   p.last_name AS teacher_last_name
FROM university.student_groups sg
JOIN university.curriculums c ON c.admission_year = EXTRACT(YEAR FROM sg.start_date)
JOIN university.discipline_in_curriculum dic ON c.curriculum_id = dic.curriculum_id
```

```
JOIN university.disciplines d ON dic.discipline_id = d.discipline_id
JOIN university.schedules sch ON dic.dis_id = sch.dis_id
JOIN university.job_history jh ON d.teacher_id = jh.history_id
JOIN university.people p ON jh.person_id = p.person_id
WHERE sg.group_number = 'K3241'
ORDER BY sch.event_date;
```

Query Query History

```
1  SELECT
2      d.name AS discipline_name,
3      sch.event_date AS exam_date,
4      p.last_name AS teacher_last_name
5  FROM university.student_groups sg
6  JOIN university.curriculums c ON c.admission_year = EXTRACT(YEAR FROM sg.start_date)
7  JOIN university.discipline_in_curriculum dic ON c.curriculum_id = dic.curriculum_id
8  JOIN university.disciplines d ON dic.discipline_id = d.discipline_id
9  JOIN university.schedules sch ON dic.dis_id = sch.dis_id
10 JOIN university.job_history jh ON d.teacher_id = jh.history_id
11 JOIN university.people p ON jh.person_id = p.person_id
12 WHERE sg.group_number = 'K3241'
13 ORDER BY sch.event_date;
```

Data Output Messages Explain × Notifications

Showing rows: 1 to 11

	discipline_name character varying (50)	exam_date date	teacher_last_name character varying (30)
1	Введение в технологическое предпринимательс...	2026-01-01	Бойцова
2	Объектно-ориентированное программирование	2026-01-03	Слюсаренко
3	Проектирование и реализация баз данных	2026-01-05	Говорова
4	Дифференциальные уравнения	2026-01-07	Лукина

2. Вывести список студентов, получивших двойки на первой попытке с указанием фамилии преподавателя, которым они должны пересдать экзамен.

```
SELECT
   p.last_name AS student_last_name,
   p.first_name AS student_first_name,
   sg.group_number,
   d.name AS discipline_name,
   sr.grade,
   tp.last_name AS teacher_last_name
FROM university.session_results sr
JOIN university.students s ON sr.student_id = s.student_id
JOIN university.people p ON s.person_id = p.person_id
JOIN university.student_groups sg ON s.group_id = sg.group_id
JOIN university.discipline_in_curriculum dic ON sr.discipline_id = dic.dis_id
JOIN university.disciplines d ON dic.discipline_id = d.discipline_id
JOIN university.job_history jh ON d.teacher_id = jh.history_id
JOIN university.people tp ON jh.person_id = tp.person_id
WHERE sr.attempt_number = 1
  AND sr.assessment_type = 'Экзамен'
  AND sr.grade < 15
ORDER BY sg.group_number, p.last_name;
```

Query Query History

```
SELECT
    p.last_name AS student_last_name,
    p.first_name AS student_first_name,
    sg.group_number,
    d.name AS discipline_name,
    sr.grade,
    tp.last_name AS teacher_last_name
FROM university.session_results sr
JOIN university.students s ON sr.student_id = s.student_id
```

Data Output Messages Explain × Notifications

Showing rows: 1 to 21 Pag

	student_last_name character varying (30)	student_first_name character varying (30)	group_number character varying	discipline_name character varying (50)	grade integer	teacher_last_name character varying (30)
1	Гуань	Андрей	К3241	Дифференциальные уравнения	7	Лукина
2	Гуань	Андрей	К3241	Проектирование взаимодействия в UX/UI дизайне	0	Кокорев
3	Гуань	Андрей	К3241	Облачные технологии и услуги	1	Меркушев
4	Гуань	Андрей	К3241	Введение в технологическое предпринимательс...	7	Бойцова
5	Гуань	Андрей	К3241	Механика	12	Курашова

3. Вывести фамилии студентов, получивших оценки по дисциплине, которые выше среднего балла по этой дисциплине.

```
SELECT DISTINCT
   p.last_name,
   p.first_name,
   d.name AS discipline_name,
   sr.grade,
   (SELECT AVG(grade)
    FROM university.session_results sr2
    WHERE sr2.discipline_id = sr.discipline_id
      AND sr2.assessment_type = 'Экзамен') AS avg_grade
FROM university.session_results sr
JOIN university.students s ON sr.student_id = s.student_id
JOIN university.people p ON s.person_id = p.person_id
JOIN university.discipline_in_curriculum dic ON sr.discipline_id = dic.dis_id
JOIN university.disciplines d ON dic.discipline_id = d.discipline_id
WHERE sr.grade > (SELECT AVG(grade)
                  FROM university.session_results sr2
                  WHERE sr2.discipline_id = sr.discipline_id
                    AND sr2.assessment_type = 'Экзамен')
ORDER BY d.name, p.last_name;
```

Query Query History

```
SELECT DISTINCT
    p.last_name,
    p.first_name,
    d.name AS discipline_name,
    sr.grade,
    (SELECT AVG(grade)
     FROM university.session_results sr2
     WHERE sr2.discipline_id = sr.discipline_id
       AND sr2.assessment_type = 'Экзамен') AS avg_grade
```

Data Output Messages Explain × Notifications

Showing rows: 1 to

	last_name character varying (30)	first_name character varying (30)	discipline_name character varying (50)	grade integer	avg_grade numeric
1	Максудов	Михаил	Введение в технологическое предпринимательс...	29	17.2000000000000000
2	Хайкин	Елизавета	Введение в технологическое предпринимательс...	26	17.2000000000000000
3	Яхина	Алексей	Введение в технологическое предпринимательс...	23	17.2000000000000000
4	Гуань	Андрей	Дифференциальные уравнения	7	5.8000000000000000

4. Создать рейтинговый список групп по успеваемости по заданному направлению по результатам сдачи сессии, упорядочив его по убыванию.

```
SELECT
   sg.group_number,
   ROUND(AVG(sr.grade), 2) AS avg_grade,
   COUNT(DISTINCT s.student_id) AS student_count
FROM university.student_groups sg
JOIN university.students s ON sg.group_id = s.group_id
JOIN university.session_results sr ON s.student_id = sr.student_id
JOIN university.curriculums c ON c.admission_year = EXTRACT(YEAR FROM sg.start_date)
JOIN university.education_programs ep ON c.program_id = ep.program_id
JOIN university.specializations sp ON ep.spec_id = sp.spec_id
WHERE sr.assessment_type = 'Экзамен'
  AND sp.spec_code = '09.03.03'
GROUP BY sg.group_number
ORDER BY avg_grade DESC;
```

Query Query History

```
1  SELECT
2      sg.group_number,
3      ROUND(AVG(sr.grade), 2) AS avg_grade,
4      COUNT(DISTINCT s.student_id) AS student_count
5  FROM university.student_groups sg
6  JOIN university.students s ON sg.group_id = s.group_id
7  JOIN university.session_results sr ON s.student_id = sr.student_id
8  JOIN university.curriculums c ON c.admission_year = EXTRACT(YEAR FROM sg.start_date)
9  JOIN university.education_programs ep ON c.program_id = ep.program_id
10 JOIN university.specializations sp ON ep.spec_id = sp.spec_id
11 WHERE sr.assessment_type = 'Экзамен'
12   AND sp.spec_code = '09.03.03'
13 GROUP BY sg.group_number
14 ORDER BY avg_grade DESC;
```

Data Output Messages Explain × Notifications

	group_number character varying	avg_grade numeric	student_count bigint
1	K3240	22.22	3
2	K3241	13.40	5

5. Создайте списки студентов, упорядоченные по группам и фамилиям студентов, содержащие данные о средних баллах и назначении на стипендии. Студент получает стипендию, если он сдал сессию без троек. Если студент не назначен на стипендию, указать 0, если назначен - 1.

```
SELECT
   sg.group_number,
   p.last_name,
   p.first_name,
   ROUND(AVG(CASE WHEN sr.assessment_type = 'Экзамен' THEN sr.grade END), 2) AS
avg_exam_grade,
   CASE
      WHEN MIN(CASE WHEN sr.status = 'Не сдано' THEN 1 ELSE 0 END) = 0
         AND MIN(CASE WHEN sr.assessment_type = 'Экзамен' AND sr.grade < 15 THEN 1
ELSE 0 END) = 0
```

```
            AND MIN(CASE WHEN sr.assessment_type = 'Зачет' AND sr.grade < 10 THEN 1 ELSE
0 END) = 0
        THEN 1 ELSE 0
    END AS scholarship_flag
FROM university.students s
JOIN university.people p ON s.person_id = p.person_id
JOIN university.student_groups sg ON s.group_id = sg.group_id
LEFT JOIN university.session_results sr ON s.student_id = sr.student_id
GROUP BY sg.group_number, p.last_name, p.first_name, s.student_id
ORDER BY sg.group_number, p.last_name;
```

Query Query History

```
1  SELECT
2      sg.group_number,
3      p.last_name,
4      p.first_name,
5      ROUND(AVG(CASE WHEN sr.assessment_type = 'Экзамен' THEN sr.grade END), 2) AS avg_exam_grade,
6      CASE
7          WHEN MIN(CASE WHEN sr.status = 'Не сдано' THEN 1 ELSE 0 END) = 0
```

Data Output Messages Explain × Notifications

Showing rows: 1 to 4

	group_number character varying	last_name character varying (30)	first_name character varying (30)	avg_exam_grade numeric	scholarship_flag integer
1	K3240	Васляев	Степан	[null]	1
2	K3240	Ведерников	Даниял	[null]	1
3	K3240	Дагаев	Дмитрий	[null]	1

6. Вывести список студентов, сдавших все положенные экзамены.

```
SELECT
    p.last_name,
    p.first_name,
    sg.group_number
FROM university.students s
JOIN university.people p ON s.person_id = p.person_id
JOIN university.student_groups sg ON s.group_id = sg.group_id
WHERE NOT EXISTS (
    SELECT 1 FROM university.session_results sr
    WHERE sr.student_id = s.student_id AND sr.status = 'Не сдано'
)
ORDER BY sg.group_number, p.last_name;
```

Query Query History

```sql
SELECT
    p.last_name,
    p.first_name,
    sg.group_number
FROM university.students s
JOIN university.people p ON s.person_id = p.person_id
JOIN university.student_groups sg ON s.group_id = sg.group_id
WHERE NOT EXISTS (
    SELECT 1 FROM university.session_results sr
```

Data Output Messages Explain × Notifications

	last_name character varying (30)	first_name character varying (30)	group_number character varying
1	Васляев	Степан	K3240
2	Ведерников	Даниял	K3240
3	Дагаев	Дмитрий	K3240
4	Лапушкина	Галина	K3240

7. Вывести список студентов, получивших максимальный средний балл в своей группе.

```sql
SELECT
   sg.group_number,
   p.last_name,
   p.first_name,
   ROUND(AVG(CASE WHEN sr.assessment_type = 'Экзамен' THEN sr.grade END), 2) AS
avg_grade
FROM university.students s
JOIN university.people p ON s.person_id = p.person_id
JOIN university.student_groups sg ON s.group_id = sg.group_id
LEFT JOIN university.session_results sr ON s.student_id = sr.student_id
GROUP BY sg.group_number, p.last_name, p.first_name, s.student_id
HAVING ROUND(AVG(CASE WHEN sr.assessment_type = 'Экзамен' THEN sr.grade END), 2)
= (
   SELECT MAX(sub_avg)
   FROM (
      SELECT
         s2.student_id,
         AVG(CASE WHEN sr2.assessment_type = 'Экзамен' THEN sr2.grade END) AS
sub_avg
      FROM university.students s2
      LEFT JOIN university.session_results sr2 ON s2.student_id = sr2.student_id
      JOIN university.student_groups sg2 ON s2.group_id = sg2.group_id
      WHERE sg2.group_number = sg.group_number
      GROUP BY s2.student_id
   ) group_avgs
)
ORDER BY sg.group_number;
```

Data Output Messages Explain × Notifications

	group_number character varying	last_name character varying (30)	first_name character varying (30)	avg_grade numeric
1	K3240	Чжоу	Семён	28.00
2	K3241	Максудов	Михаил	16.00

Создание представлений

1. Список студентов, получивших двойки на первой попытке с указанием фамилии преподавателя, которым они должны пересдать экзамен;

```
CREATE VIEW university.fail_on_first_attempt AS
SELECT
   p.last_name AS student_last_name,
   p.first_name AS student_first_name,
   sg.group_number,
   d.name AS discipline_name,
   sr.grade,
   tp.last_name AS teacher_last_name
FROM university.session_results sr
JOIN university.students s ON sr.student_id = s.student_id
JOIN university.people p ON s.person_id = p.person_id
JOIN university.student_groups sg ON s.group_id = sg.group_id
JOIN university.discipline_in_curriculum dic ON sr.discipline_id = dic.dis_id
JOIN university.disciplines d ON dic.discipline_id = d.discipline_id
JOIN university.job_history jh ON d.teacher_id = jh.history_id
JOIN university.people tp ON jh.person_id = tp.person_id
WHERE sr.attempt_number = 1
  AND sr.assessment_type = 'Экзамен'
  AND sr.grade < 15;
```

Query Query History

```
1  SELECT * FROM university.fail_on_first_attempt
```

Data Output Messages Explain × Notifications

Showing rows: 1 to 21 Pag

	student_last_name character varying (30)	student_first_name character varying (30)	group_number character varying	discipline_name character varying (50)	grade integer	teacher_last_name character varying (30)
1	Шнейдерис	Пётр	K3241	Теория вероятностей	12	Блейхер
2	Шнейдерис	Пётр	K3241	Облачные технологии и услуги	4	Меркушев
3	Шнейдерис	Пётр	K3241	Дифференциальные уравнения	13	Лукина
4	Шнейдерис	Пётр	K3241	Введение в технологическое предпринимательс...	1	Бойцова
5	Гуань	Андрей	K3241	Механика	12	Кувшова

2. Данные о студентах при получении ими хотя бы одной оценки 2 (после 3-й попытки).

```
CREATE VIEW university.failed_after_third_attempt AS
SELECT DISTINCT
   p.last_name,
   p.first_name,
   sg.group_number,
   d.name AS discipline_name,
```

```
    sr.attempt_number,
    sr.grade,
    tp.last_name AS teacher_last_name
FROM university.session_results sr
JOIN university.students s ON sr.student_id = s.student_id
JOIN university.people p ON s.person_id = p.person_id
JOIN university.student_groups sg ON s.group_id = sg.group_id
JOIN university.discipline_in_curriculum dic ON sr.discipline_id = dic.dis_id
JOIN university.disciplines d ON dic.discipline_id = d.discipline_id
JOIN university.job_history jh ON d.teacher_id = jh.history_id
JOIN university.people tp ON jh.person_id = tp.person_id
WHERE sr.attempt_number >= 3
  AND sr.assessment_type = 'Экзамен'
  AND sr.grade < 15;
```

Query Query History

```
1  SELECT * FROM university.failed_after_third_attempt
```

Data Output Messages Explain × Notifications

Showing rows: 1 to 4 Page No: 1

	last_name character varying (30)	first_name character varying (30)	group_number character varying	discipline_name character varying (50)	attempt_number integer	grade integer	teacher_last_name character varying (30)
1	Гуань	Андрей	K3241	Техники публичных выступлений и презента...	3	5	Тихомирова
2	Максудов	Михаил	K3241	Механика	3	10	Курашова
3	Шнейдерис	Пётр	K3241	Жизненный цикл программного обеспечения	3	8	Осетрова
4	Яхина	Алексей	K3241	Объектно-ориентированное программирован...	3	6	Слюсаренко

Запросы на модификацию

1. INSERT - новая стипендия (спортивная), назначаем ее студентам K3241, у которых нет двоек.

```
INSERT INTO university.scholarships (type, amount, conditions)
VALUES ('Спортивная', 3000.00, 'Участие в спортивных соревнованиях')
```

Query Query History

```
1  SELECT * FROM university.scholarships
```

Data Output Messages Explain × Notifications

	scholarship_id [PK] integer	type character varying (100)	amount numeric (10,2)	conditions text
1	1	Академическая	2300.00	Успеваемость без троек, сессия на "4" и "5"
2	2	Повышенная академическ...	4300.00	Только отличники (все оценки "5")
3	3	Президентская	7000.00	Победители олимпиад, особые научные дост...
4	4	Именная стипендия	6000.00	Участие в проектах, хакатонах, научных рабо...
5	5	Спортивная	3000.00	Участие в спортивных соревнованиях

```
INSERT INTO university.scholarship_assignments (student_id, scholarship_id,
assignment_date, is_active)
SELECT
    s.student_id,
    (SELECT scholarship_id FROM university.scholarships WHERE type = 'Спортивная'),
    CURRENT_DATE,
    true
FROM university.students s
JOIN university.student_groups sg ON s.group_id = sg.group_id
```

```
WHERE sg.group_number = 'K3241'
  AND NOT EXISTS (
    SELECT 1 FROM university.session_results sr
    WHERE sr.student_id = s.student_id AND sr.status = 'Не сдано'
  );
```

Query Query History

```
1  SELECT * FROM university.scholarship_assignments
```

Data Output Messages Explain × Notifications

	assignment_id [PK] integer	student_id integer	scholarship_id integer	assignment_date date	is_active boolean
33	33	33	2	2026-02-01	true
34	34	34	3	2025-09-01	true
35	35	35	2	2025-09-01	true
36	36	36	1	2025-09-01	true
37	37	37	4	2026-02-01	false
38	38	38	3	2025-09-01	true

2. UPDATE - сделать стипендию неактивной в случае, если есть несданные экзамены/зачеты.

```
SELECT sa.student_id, sa.is_active, sr.status
FROM university.scholarship_assignments sa
JOIN university.session_results sr ON sa.student_id = sr.student_id
WHERE sr.status = 'Не сдано'
```

Query Query History

```
1  SELECT sa.student_id, sa.is_active, sr.status
2  FROM university.scholarship_assignments sa
3  JOIN university.session_results sr ON sa.student_id = sr.student_id
4  WHERE sr.status = 'Не сдано'
```

Data Output Messages Explain × Notifications

	student_id integer	is_active boolean	status character varying
1	1	true	Не сдано
2	1	true	Не сдано
3	1	true	Не сдано
4	1	true	Не сдано
5	1	true	Не сдано
6	1	true	Не сдано

```
UPDATE university.scholarship_assignments
```

```
SET is_active = false
WHERE student_id IN (
   SELECT DISTINCT student_id
   FROM university.session_results
   WHERE status = 'Не сдано'
)
AND is_active = true;
```

Query Query History

```
1  SELECT sa.student_id, sa.is_active, sr.status
2  FROM university.scholarship_assignments sa
3  JOIN university.session_results sr ON sa.student_id = sr.student_id
4  WHERE sr.status = 'Не сдано'
```

Data Output Messages Explain × Notifications

	student_id integer	is_active boolean	status character varying
1	1	false	Не сдано
2	1	false	Не сдано
3	1	false	Не сдано
4	1	false	Не сдано
5	1	false	Не сдано
6	1	false	Не сдано

3. DELETE - удалить неактивные назначения стипендий старше 6 месяцев.

```
SELECT
   sa.assignment_id,
   sa.student_id,
   p.last_name,
   p.first_name,
   sa.scholarship_id,
   s.type AS scholarship_type,
   sa.assignment_date,
   sa.is_active
FROM university.scholarship_assignments sa
JOIN university.students st ON sa.student_id = st.student_id
JOIN university.people p ON st.person_id = p.person_id
JOIN university.scholarships s ON sa.scholarship_id = s.scholarship_id
WHERE sa.student_id IN (
   SELECT DISTINCT student_id
   FROM university.session_results
   WHERE status = 'Не сдано'
)
AND sa.assignment_date < CURRENT_DATE - INTERVAL '6 months'
AND sa.is_active = false
ORDER BY sa.student_id, sa.assignment_date;
```

Data Output Messages Explain × Notifications

Showing rows: 1 to 2 Page

	assignment_id integer	student_id integer	last_name character varying (30)	first_name character varying (30)	scholarship_id integer	scholarship_type character varying (100)	assignment_date date	is_active boolean
1	1	1	Шнейдерис	Пётр	3	Президентская	2025-09-01	false
2	3	3	Максудов	Михаил	4	Именная стипендия	2025-09-01	false

```
DELETE FROM university.scholarship_assignments
WHERE student_id IN (
   SELECT student_id
   FROM university.session_results
   WHERE status = 'Не сдано'
)
AND assignment_date < CURRENT_DATE - INTERVAL '6 months'
AND is_active = false;
```

Query Query History

```
SELECT
    sa.assignment_id,
    sa.student_id,
    p.last_name,
    p.first_name,
    sa.scholarship_id,
    s.type AS scholarship_type,
    sa.assignment_date,
    sa.is_active
FROM university.scholarship_assignments sa
JOIN university.students st ON sa.student_id = st.student_id
JOIN university.people p ON st.person_id = p.person_id
JOIN university.scholarships s ON sa.scholarship_id = s.scholarship_id
```

Data Output Messages Explain × Notifications

	assignment_id integer	student_id integer	last_name character varying (30)	first_name character varying (30)	scholarship_id integer	scholarship_type character varying (100)	assignment_date date	is_active boolean

Создание индексов

Запрос 1 (без индекса) - поиск человека с фамилией Шнейдерис.

```
EXPLAIN ANALYZE
SELECT * FROM university.people WHERE last_name = 'Шнейдерис'
```

Query Query History

```
EXPLAIN ANALYZE
SELECT * FROM university.people WHERE last_name = 'Шнейдерис'
```

Data Output Messages Explain × Notifications

	QUERY PLAN text
1	Seq Scan on people (cost=0.00..1.50 rows=1 width=63) (actual time=0.027..0.032 rows=1.00 loop...
2	Filter: ((last_name)::text = 'Шнейдерис'::text)
3	Rows Removed by Filter: 50
4	Buffers: shared hit=1
5	Planning Time: 0.116 ms
6	Execution Time: 0.053 ms

Data Output | Messages | Explain ✕ | Notifications

```
Successfully run. Total query runtime: 59 msec.
6 rows affected.
```

Запрос 2 (без индекса) - подсчет количества экзаменов для каждого студента

```
EXPLAIN ANALYZE
SELECT student_id, COUNT(*)
FROM university.session_results
WHERE assessment_type = 'Экзамен'
GROUP BY student_id
```

Query | Query History

```
1 EXPLAIN ANALYZE
2 SELECT student_id, COUNT(*)
3 FROM university.session_results
4 WHERE assessment_type = 'Экзамен'
5 GROUP BY student_id
```

Data Output | Messages | Explain ✕ | Notifications

	QUERY PLAN text
1	HashAggregate (cost=1.86..1.91 rows=5 width=12) (actual time=0.036..0.038 rows=8.00 loops=1)
2	Group Key: student_id
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=1
5	-> Seq Scan on session_results (cost=0.00..1.69 rows=35 width=4) (actual time=0.017..0.023 rows=48.00 loo...
6	Filter: ((assessment_type)::text = 'Экзамен'::text)
7	Rows Removed by Filter: 20
8	Buffers: shared hit=1
9	Planning Time: 0.103 ms
10	Execution Time: 0.078 ms

Query | Query History

```
1 EXPLAIN ANALYZE
2 SELECT student_id, COUNT(*)
3 FROM university.session_results
4 WHERE assessment_type = 'Экзамен'
5 GROUP BY student_id
```

Data Output | Messages | Explain ✕ | Notifications

```
Successfully run. Total query runtime: 56 msec.
10 rows affected.
```

Создание индексов

```
CREATE INDEX idx_people_last_name ON university.people (last_name);
CREATE INDEX idx_session_results_student_assessment
ON university.session_results (student_id, assessment_type);
```

Запрос 1 (с индексом)

Query Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM university.people WHERE last_name = 'Шнейдерис'
```

Data Output Messages Explain × Notifications

	QUERY PLAN text
1	Seq Scan on people (cost=0.00..1.64 rows=1 width=63) (actual time=0.012..0.014 rows=1.00 loop...
2	Filter: ((last_name)::text = 'Шнейдерис'::text)
3	Rows Removed by Filter: 50
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=16 read=1 dirtied=2
7	Planning Time: 1.543 ms
8	Execution Time: 0.029 ms

Query Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM university.people WHERE last_name = 'Шнейдерис'
```

Data Output Messages Explain × Notifications

```
Successfully run. Total query runtime: 59 msec.
6 rows affected.
```

Запрос 2 (с индексом)

Query Query History

```
1  EXPLAIN ANALYZE
2  SELECT student_id, COUNT(*)
3  FROM university.session_results
4  WHERE assessment_type = 'Экзамен'
5  GROUP BY student_id
```

Data Output Messages Explain × Notifications

	QUERY PLAN text
1	HashAggregate (cost=2.06..2.11 rows=5 width=12) (actual time=0.032..0.034 rows=8.00 loops=1)
2	Group Key: student_id
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=1
5	-> Seq Scan on session_results (cost=0.00..1.85 rows=43 width=4) (actual time=0.015..0.019 rows=48.00 loo...
6	Filter: ((assessment_type)::text = 'Экзамен'::text)
7	Rows Removed by Filter: 20
8	Buffers: shared hit=1
9	Planning:
10	Buffers: shared hit=23 read=1
11	Planning Time: 1.096 ms
12	Execution Time: 0.067 ms

Query Query History

```
1 EXPLAIN ANALYZE
2 SELECT student_id, COUNT(*)
3 FROM university.session_results
4 WHERE assessment_type = 'Экзамен'
5 GROUP BY student_id
```

Data Output Messages Explain × Notifications

```
Successfully run. Total query runtime: 62 msec.
12 rows affected.
```

Удаление индексов

DROP INDEX university.idx_people_last_name;
DROP INDEX university.idx_session_results_student_assessment;

Query Query History

```
1 DROP INDEX university.idx_people_last_name;
2 DROP INDEX university.idx_session_results_student_assessment;
```

Data Output Messages Explain × Notifications

```
DROP INDEX

Query returned successfully in 65 msec.
```

Выводы

При выполнении лабораторной работы были освоены практические навыки создания сложных запросов на выборку данных с использованием JOIN, агрегатных функций и подзапросов, а также создания представлени. Были составлены и выполнены три запроса на модификацию данных (INSERT, UPDATE, DELETE) с вложенными подзапросами.